

Alternative Language Support in Smalltalk Image

Adrien Hopkins¹, Dave Mason¹

¹Toronto Metropolitan University, Toronto, Canada

Abstract

Smalltalk systems have three primary components: syntax, semantics, and the image. Implementing the semantics in a runtime or virtual machine is a major engineering task and a requirement for the other two. However, the syntax and the image are, in some sense less fundamental. GNU Smalltalk demonstrates that the syntax and semantics can be divorced from a requirement for an image and remain useful. This paper considers divorcing the other end - the Smalltalk syntax.

Smalltalk systems have many unique features and advantages, such as being based on an extremely simple language; however, they are not widely used or known. The Zag Smalltalk system stores code as abstract syntax trees (ASTs). We are exploring how to expand the utility of Smalltalk by translating Smalltalk code to and from other languages, particularly Python. This paper describes a prototype that can translate basic programs between Python and Zag Smalltalk. To further our experiments we further translate code from the Zag AST into Pharo Smalltalk's Opal AST, which allows testing execution of this translated code in Pharo. This work is in the early phases of construction, so many features remain unimplemented, some of which pose unique challenges.

1. Introduction

While Smalltalk syntax is beloved by many (including the authors of this paper), there are a variety of reasons why it might be useful to support alternatives:

Unfamiliarity. Most developers have no experience with Smalltalk and often think it “looks weird” and therefore miss out on the highly productive Smalltalk environment.

Community size. While “many” like Smalltalk syntax, “many” isn’t as many as we might like, and managers are often willing to ignore Smalltalk’s benefits because of unfamiliarity or fear of being unable to find developers.

LLM Support. While LLMs are somewhat language agnostic and seem to do Smalltalk quite well, the available training data for e.g. Python is vastly larger than that for Smalltalk.

Library support. Many popular and powerful libraries are implemented in Python, such as Pandas[1] for data management, Matplotlib[2] for data visualization, and TensorFlow[3] for machine learning. Translation is one way these libraries could be used in Smalltalk, in addition to foreign function interfaces.

The Zag Smalltalk system[4, 5] does not store text for methods, but rather annotated abstract syntax trees (ASTs). This means that a language that is semantically close enough to Smalltalk could be translated from the source language to a Zag AST. Furthermore, an AST could be displayed in alternative source languages. In principle, this means that each developer could develop and debug in the language of their choice. This is a more integrated approach than that in [6]. As opposed to the interpreter-generation approach of [7], this is an attempt to support multiple languages in a single runtime.

The end goal for this work is for the user to be able to, in the image, choose a language in which they want to interact. If they choose Smalltalk, they will see the conventional representation of the method, whether in the browser or a debugger. If they choose another language, such as Python, they will see the same series of windows, but the actual code would show the Python equivalent. When looking at a method, if the language the user has selected is the same as the language it was entered in, they should



see exactly what was entered. If the language is different there may be some artifacts, although the long-term goal is to minimize that.

2. Related Work

Much work has already been done in the field of programming language translation. There are two primary approaches people have used to translate programming languages: manually creating translation rules, and translating using machine learning.

The first form of code translation is done by applying manually created transformation rules [8, 9]. This requires creating a large number of manual rules [10, 11], particularly for APIs [12], and creates challenges when translating between different programming paradigms [13, 14, 11]. These manually-generated rules will need to be different for each pair of programming languages that a system can translate between, as every language has different semantics [13]. However, perfect functional equivalence between paradigms may not be desirable if it leads to overly complicated code [15, 16].

Another technique is to use machine learning (ML) systems based on those used to translate human languages [17, 18]. However, this has the problem that small changes in code can significantly change behaviour [19, 20, 16, 17]. For example, changing a few statements in 3 lines of code of a sorting algorithm can turn it into an entirely different sorting algorithm [20]. It will also be important to have access to large datasets of code for training [21, 22], which can be difficult to find for less popular languages [22]. Unsupervised methods, such as training AI to fill in missing parts of a partially implemented function, can address the drawbacks of this approach [16, 22, 11]. Another technique is to use abstract syntax trees as an intermediate representation, since parts of programs can be effectively represented as subtrees [8, 23]. In addition, systems designed to fix errors in ML-generated code could be used for ML-generated translation outputs [20, 24].

One example of a system similar to ours is PharoJS [25], a system that translates Pharo Smalltalk code into JavaScript to allow Pharo code to be run in JavaScript. Another example is MagLev [26], which explored the implementation of Ruby[27] on the Gemstone/S[28] Smalltalk VM. A third example is Gemstone’s Grail system [29], which translates Python code into Smalltalk to execute Python code in a Smalltalk system. However, it does not support code translation from Smalltalk to Python.

3. Python and Smalltalk

Python[30, 31] is by far the most popular dynamically-typed language at the time of writing, with many useful libraries available for various applications. Therefore, it is a natural candidate for support in our system.

Python has major similarities with Smalltalk, which makes translation easier than with most other language pairs:

- Python and Smalltalk are both dynamically typed. If one of the languages were statically typed, then translation would require inferring the types of every variable, or casting types in order to be able to call methods.
- Every value in both Python and Smalltalk is an object that exists within the language’s class hierarchy.

Therefore, the first part of our multi-language translation system is a system that can translate from Python to Zag and Zag to Python.

3.1. From Python to Smalltalk

The implementation of Python-to-Smalltalk translation is relatively straight-forward. Currently we are using tree-sitter[32] to parse the Python source into a Python AST. Then we use a recursive tree walk

to translate the tree-sitter AST to the Zag (Smalltalk) AST. We may switch to a bespoke parser in future, but tree-sitter makes it easy to quickly build a prototype.

The tree-sitter AST only describes the structure of the program; it does not include details such as the names of variables or the specific operator that a node represents. The only instance variables that tree-sitter nodes have are:

- the type of the node, such as "if statement" or "binary operator", and
- the start and end positions of this node in the source code.

In order to parse the tree-sitter AST, we created our own node objects that contain both the tree-sitter node and a copy of the source code. Details like variable names and specific operators can then be easily obtained by reading the node's part of the source code, sometimes involving basic parsing.

Although Python is similar to Smalltalk, there are many interesting challenges when translating from Python to Smalltalk:

- Python uses truthy/falsey boolean values, so whenever a boolean test is being performed, a special message called `toPythonBoolean` must be sent before messages such as `ifTrue:ifFalse:.` This method is used to minimize the number of new messages that had to be added, given that we intend to translate multiple languages, several of which have their own truthiness semantics. When looking at the method through a Smalltalk lens, this will be visible, but it also means this method is available to Smalltalk code.
- Boolean operators always return one of their arguments, meaning that "1 and 1" is not the same as "True and True", even though 1 is truthy. This behaviour is useful, and should be kept in translated code, but is not possible to replicate using `toPythonBoolean`. To solve this, Python's `and` and `or` operators are translated into the special messages `pythonAnd:` and `pythonOr:`.
- Python conditionals and loops will be translated to the appropriate message sends. When looking at the method through a Python lens, we must recognize the pattern and turn it back into an `if` or `while` statement.

A custom switch statement was added as an extension, so long `if/elif` chains can be translated without excessive nesting, which would be required if using `ifTrue:ifFalse:.` It is only used if the Python code has an `elif` statement, otherwise a pre-existing Smalltalk `if` message - either `ifTrue:` or `ifTrue:ifFalse:-` is used.

Using this custom switch statement, the following Python code:

```
if x == y:
    print("x and y are equal")
elif y == z:
    print("y and z are equal")
elif x == z:
    print("x and z are equal")
else:
    print("none of them are equal")
```

can be translated into the following Smalltalk:

```
{
  x = y -> [ Transcript show: "x and y are equal"; cr ].
  y = z -> [ Transcript show: "y and z are equal"; cr ].
  x = z -> [ Transcript show: "x and z are equal"; cr ].
  true -> [ Transcript show: "none of them are equal"; cr ].
} switch
```

The source code of this switch method, excluding comments and examples, is:

```

Collection>>switch
  ^self
  detect: [ :each | each key value ]
  ifFound: [ :each | each value value ]
  ifNone: nil.

```

- Python supports the `break` and `continue` statements, which can be used to end a loop iteration early. Smalltalk has no such functionality. While they can be worked around with conditional blocks, the result is ugly and non-idiomatic. Alternatively, both will need a special version of `return` to exit from the appropriate block.
- Python uses multiple inheritance, but Smalltalk classes only support single inheritance. This will be implemented by having translated classes inherit from traits instead of ordinary classes. There will have to be a way to display a trait if it implements a more exotic inheritance structure.
- Primitive methods like `__add__` have fixed arity so they can be translated directly to use the Smalltalk `+` methods.
- Regular methods can be variadic and have an optional dictionary as the last parameter, so a new `send` mechanism must be supported. This will be implemented by having special parameters that accept an array of variadic arguments and/or a dictionary of keyword arguments. These parameters would need to be distinguished from normal parameters by either adding a special property to the Zag AST that states which parameters are variadic or by using a naming convention.
- Python functions can be defined inside other functions, and returning returns from the innermost function. Smalltalk methods cannot be defined inside other methods. These are essentially named Smalltalk blocks, which can be defined inside methods or other blocks, but they will need the revised return mentioned above.
- Arrays are indexed from 0 instead of 1, and negative indices can be used to access elements near the end of an array (`-1` = last, `-2` = second-last, etc.). This was solved by adding custom messages such as `zwAt:`, which is like `at:` but with Python's zero-indexed index-wrapping behaviour. Then the array index operation can be translated to `zwAt:` instead of `at:`, and similarly for other index-requiring operations. Two of these operations are currently used, as shown in table 1.

Python construct	Custom Smalltalk equivalent
<code>a[x]</code>	<code>a zwAt: x</code>
<code>a[x] = y</code>	<code>a zwAt: x put: y</code>

Table 1

The basic array statements in Python, and their translated versions in Smalltalk

Though the system does not yet support translating arbitrary library calls, some specific library calls were given manual translation rules to make it possible to test the system with fully functional programs. These calls are listed in table 2.

Python call	Smalltalk translation
<code>print(x)</code>	<code>Transcript show: x; cr</code>
<code>input(x)</code>	<code>UIManager default request: x</code>
<code>int(x)</code>	<code>x asInteger</code>
<code>float(x)</code>	<code>x asNumber</code>
<code>str(x)</code>	<code>x asString</code>

Table 2

The currently supported Python standard library calls, and their translations in Smalltalk

These library calls were added as they became necessary to translate simple programs.

3.2. From Smalltalk to Python

Though not necessary to bring Python libraries into Smalltalk or translate LLM-generated Python, translating from Smalltalk to Python is necessary for our end goal of having a system where users are able to view and edit programs in their preferred language. Therefore, conversion from Smalltalk to Python is included in our project.

We translate from Zag to Python directly, without any intermediate AST. All Zag node types except `ASSend` and `ASCascade` represent simple constructs like literals, variables, and return statements that can simply be translated directly into the corresponding source code. `ASCascade` represents a cascade statement, so it can be simply translated as if it were multiple sends.

`ASSend` is the only complex statement to translate, and it is simply translated by performing the Python-to-Zag translation rules in reverse. For example, a Python `for-in` loop is translated into a Zag `do: send`, therefore when translating in reverse a `do: send` is translated into a `for-in` loop. All the examples in section 3.1 apply in reverse to our Zag-to-Python translation.

3.3. Lessons Learned

There are major syntactic differences between Python and Smalltalk, but translation of syntax is relatively easy. The most significant challenge in programming language translation is translating in a way that accurately reflects the different semantics of Python and Smalltalk.

Smalltalk has one major useful mechanism to allow for the handling of semantic differences, extension messages. These allow for systems to add messages to pre-existing objects, which can be used to add new features to the Smalltalk language. Because of extension messages, everything we have done so far has been able to be translated into the pre-existing Zag AST, without needing to add any features that do not exist in other Smalltalk systems. For example, `toPythonBoolean` is used to translate Python's truthiness semantics, while `zwAt:` and `zwAt:put:` are used to translate Python's array indexing semantics. When translating from Zag to Python, these messages can simply be translated back into the original Python features. Default Smalltalk messages need a more complex translation, such as translating the Smalltalk code `x at: i` to the Python code `x[i - 1]`.

Some of the challenges mentioned above are not directly translatable into the Zag Smalltalk AST, and will need extensions such as local returns in blocks.

4. Zag and Opal

While the target for this work is the Zag Smalltalk system, it currently does not support a graphical user interface, so we have added the capability to generate Opal[33] ASTs from any Zag AST. This is only needed temporarily in order to run generated Zag programs; once the Zag system is fully functional, programs will be run directly from the Zag AST without any need for Opal. In this section we describe how that works.

The implementation is fairly similar to the implementation of the Python-to-Zag converter described in the previous section. Given that the Zag AST and Opal AST are both Smalltalk ASTs, they have incredibly similar structures. Hence, this system is much simpler than the Python-to-Zag converter; all the conversion methods are one or two lines of code excluding variable declarations.

Both of these ASTs are simply Smalltalk objects, with a very similar class structure, described in table 3. Complex Smalltalk constructs are implemented by having AST objects contain other AST objects as instance variables. For example, `ASBlockNode/OCBlockNode` and `ASMethodNode/OCMethodNode` have instance variables that contain the AST of the blocks' and methods' body statements.

Every element of the Zag AST is a subclass of the `AST` class. They all share an `asPharoAST` message. The Zag and Opal ASTs have very similar structure, so all this method does is return the associated Opal class or object, with equivalent instance variables. For more complicated objects that contain AST parts as instance variables, `asPharoAST` is recursively called on these parts to get converted

Zag Class	Opal Class/Object	Constructs represented
AST	OCNode	Superclass of all AST nodes
ASBlockNode	OCBlockNode	Block closures
ASMethodNode	OCMethodNode	Message definitions
ASPragma	OCPragmaNode	Pragmas
ASArray	OCArrayNode	Array objects defined with {}
ASAssign	OCAssignmentNode	Assignment expressions
ASCascade	OCCascadeNode	Cascade expressions
ASLiteral	OCLiteralNode	Literal values
ASTThisContext	OCVariableNode thisContextNode	This context
ASTThisProcess	OCVariableNode thisProcessNode	This process
ASRef	OCVariableNode	Variables
ASSend	OCMessageNode	Message sends
ASReturn	OCReturnNode	Return statements

Table 3

A list of classes in the Zag and Opal ASTs, and what they represent [5, 34]

instance variables. In a few cases, slight changes are needed to the instance variables to reflect different structures.

Once you have an Opal AST, you can execute the generated AST as a Smalltalk program. The Opal compiler is able to accept an AST via the code `OpalCompiler new ast: myAST`, and it can execute this code by sending this compiler object the `evaluate` message. Therefore, Python code can be run in the Pharo system by first translating the Python code into a Python AST using tree-sitter, then translating the Python AST into an equivalent Zag AST using the system described in section 3.1, then translating the Zag AST into an equivalent Opal AST using the system described in this section, then running the Opal AST using Pharo's Opal compiler.

5. Evaluation

5.1. Present Evaluation Procedures

Currently, a small amount of testing and evaluation has been done for the system, as it is still in the construction phase. However, as of the writing of this paper, we have 87 unit tests for the tree-sitter to Zag converter, including 2 tests that test converting small, functional programs from Python source code to the Zag AST. They are:

- a simple, three-line Fahrenheit to Celsius converter, and
- a very barebones guessing game.

All automatic tests are run before making a commit to this project's git repository, and changes are only committed if all automatic tests pass.

We also manually tested a full conversion of the Fahrenheit to Celsius converter as per the method described in section 4, and the program executed successfully.

The Python code for this converter is:

```
tempF = float(input("Input temp in F: "))
tempC = (tempF - 32) / 1.8
print(tempC)
```

It is translated into the following Smalltalk:

```
| tempF tempC |
tempF := (UIManager default request: 'Input temp in F: ') asNumber.
tempC := tempF - 32 / 1.8.
Transcript show: tempC; cr.
```

5.2. Future Evaluation Procedures

We will continue to add automatic tests, including more full programs, as the system is further developed. In addition, once the system is more complete, we will perform more extensive testing.

Our first criterion for the system’s completeness is the proportion of Python features that can be translated to and from Python. These features are defined as sections in the Python specification [30] that do not have subsections, in order to have an unbiased list. Once we are at or near 100% feature support, we can begin more rigorously testing the translations’ accuracy.

Our main idea for testing the system in this way is generating multiple kinds of programs to solve the same problem, and ranking the methods in terms of program accuracy. The types of programs we will test include:

- Python programs generated by LLMs or written manually by a skilled programmer,
- Smalltalk programs generated by the same source as the Python programs, and
- programs generated in Python then translated to Smalltalk using our transpilation system.

We expect the non-translated Python programs to perform the best; they are included as the control for this experiment. Our system will be considered to work very well if the translated Smalltalk programs are more accurate than LLM-generated Smalltalk programs.

We have not decided how to determine the accuracy of the generated programs. Some metrics we are considering are:

- the proportion of generated programs that compile successfully,
- the proportion of test cases passed by generated programs,
- the amount of time it takes for a skilled programmer to fix programs such that they pass all test cases, and
- the similarity between generated and fixed programs.

As our system is not yet complete, the specific evaluation procedures are subject to change.

6. Conclusions

Programming language translation is an interesting and useful field with several applications, including aiding LLM generation and allowing programs of different languages to be combined. As part of the Zag Smalltalk system, we are in the early stages of implementing a system that allows the same code to be represented as either Smalltalk or Python, and freely translated between the two. We have already successfully translated basic programs from Python to Smalltalk and from Smalltalk to Python, and once the system is more developed, the system will be able to translate arbitrary programs between Python and Smalltalk.

Much work remains in the future: completing Python-to-Smalltalk and Smalltalk-to-Python translation for all language features, translating library calls, and more. Many more challenges are expected as we complete the system and support translating more types of programs.

References

- [1] Pandas authors, Pandas, 2026. URL: <https://pandas.pydata.org/>.
- [2] Matplotlib authors, Matplotlib: Visualization with python, 2026. URL: <https://matplotlib.org/>.
- [3] TensorFlow authors, Tensorflow, 2026. URL: <https://github.com/tensorflow/tensorflow>.
- [4] D. Mason, Design principles for a high-performance Smalltalk, in: International Workshop on Smalltalk Technologies (IWST 2022), 2022. URL: <https://sarg.torontomu.ca/dmason/UKSt2022-11-30.pdf>.
- [5] D. Mason, Zag smalltalk, <https://github.com/Zag-Research/Zag-Smalltalk>, 2025. Accessed: 2025-04-08.

- [6] F. Niephaus, T. Felgentreff, T. Pape, R. Hirschfeld, Squeak makes a good python debugger: Bringing other programming languages into smalltalk's tools, in: Companion Proceedings of the 1st International Conference on the Art, Science, and Engineering of Programming, Programming '17, Association for Computing Machinery, New York, NY, USA, 2017. URL: <https://doi.org/10.1145/3079368.3079402>. doi:10.1145/3079368.3079402.
- [7] C. F. Bolz, A. Kuhn, A. Lienhard, N. D. Matsakis, O. Nierstrasz, L. Renggli, A. Rigo, T. Verwaest, Back to the future in one week — implementing a smalltalk vm in pypy, in: R. Hirschfeld, K. Rose (Eds.), Self-Sustaining Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 123–139.
- [8] M. Rabinovich, M. Stern, D. Klein, Abstract Syntax Networks for Code Generation and Semantic Parsing, 2017. URL: <http://arxiv.org/abs/1704.07535>. doi:10.48550/arXiv.1704.07535, arXiv:1704.07535 [cs].
- [9] R. S. Ledley, J. B. Wilson, Automatic programming language translation through syntactical analysis, Communications of the ACM 5 (1962) 145–155.
- [10] K. An, N. Meng, E. Tilevich, Automatic inference of Java-to-swift translation rules for porting mobile applications, ACM, New York, NY, USA, 2018, pp. 180–190. URL: <https://ieeexplore.ieee.org/document/8543453>. doi:10.1145/3197231.3197240, book Title: 2018 IEEE ACM 5th International Conference on Mobile Software Engineering and Systems (MOBILESoft).
- [11] B. Roziere, M.-A. Lachaux, L. Chanussot, G. Lample, Unsupervised translation of programming languages, Advances in neural information processing systems 33 (2020) 20601–20611.
- [12] H. Zhong, S. Thummalapenta, T. Xie, L. Zhang, Q. Wang, Mining API mapping for language migration, volume 1, ACM, New York, NY, USA, 2010, pp. 195–204. URL: <https://ieeexplore.ieee.org/document/6062087>. doi:10.1145/1806799.1806831, book Title: 2010 ACM/IEEE 32nd International Conference on Software Engineering.
- [13] L. Qiu, Programming language translation, Technical Report, Cornell University, 1999.
- [14] K. Yasumatsu, N. Doi, SPiCE: a system for translating Smalltalk programs into a C environment, IEEE transactions on software engineering 21 (1995) 902–912. URL: <https://ieeexplore.ieee.org/document/473219>. doi:10.1109/32.473219, place: New York, NY.
- [15] M. Mossienko, Automated Cobol to Java recycling, in: Seventh European Conference on Software Maintenance and Reengineering, 2003. Proceedings., 2003, pp. 40–50. URL: <https://ieeexplore.ieee.org/document/1192409>. doi:10.1109/CSMR.2003.1192409.
- [16] B. Roziere, J. Zhang, F. Charton, M. Harman, G. Synnaeve, G. Lample, Leveraging Automated Unit Tests for Unsupervised Code Translation, 2021. URL: <https://openreview.net/forum?id=cmt-6KtR4c4>.
- [17] A. T. Nguyen, T. T. Nguyen, T. N. Nguyen, Lexical statistical machine translation for language migration, ACM, New York, NY, USA, 2013, pp. 651–654. doi:10.1145/2491411.2494584, book Title: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering.
- [18] S. Karaivanov, V. Raychev, M. Vechev, Phrase-based statistical translation of programming languages, in: Proceedings of the 2014 ACM international symposium on new ideas, new paradigms, and reflections on programming & software, 2014, pp. 173–184.
- [19] AVATAR: A Parallel Corpus for Java-Python Program Translation, ??? URL: <https://aclanthology.org/2023.findings-acl.143.pdf>. doi:10.18653/v1/2023.findings-acl.143, book Title: Findings of the Association for Computational Linguistics: ACL 2023.
- [20] K. Wang, R. Singh, Z. Su, Dynamic Neural Program Embedding for Program Repair, 2018. URL: <http://arxiv.org/abs/1711.07163>. doi:10.48550/arXiv.1711.07163, arXiv:1711.07163 [cs].
- [21] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker, V. Thost, L. Buratti, S. Pujar, S. Ramji, U. Finkler, S. Malaika, F. Reiss, CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks, 2021. URL: <https://openreview.net/forum?id=6vZVBkCDrHT>.
- [22] F. Liu, J. Li, L. Zhang, Syntax and Domain Aware Model for Unsupervised Program Translation, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 755–767. URL: <https://ieeexplore.ieee.org/document/10172589>. doi:10.1109/ICSE48619.2023.00072.
- [23] X. Chen, C. Liu, D. Song, Tree-to-tree Neural Networks for Program Translation, 2018. URL:

- <http://arxiv.org/abs/1802.03691>. doi:10.48550/arXiv.1802.03691, arXiv:1802.03691 [cs].
- [24] M. Allamanis, E. T. Barr, C. Bird, C. Sutton, Learning natural coding conventions, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, ACM, Hong Kong China, 2014, pp. 281–293. URL: <https://dl.acm.org/doi/10.1145/2635868.2635883>. doi:10.1145/2635868.2635883.
 - [25] PharoJS contributors, Pharojs: Develop in pharo, run on javascript, ??? URL: <https://pharojs.org/>, accessed: 2026-08-14.
 - [26] M. Springer, Inter-language collaboration in an object-oriented virtual machine, 2016. URL: <https://arxiv.org/abs/1606.03644>. arXiv:1606.03644.
 - [27] D. Flanagan, Y. Matsumoto, The Ruby Programming Language, O'Reilly Media, Sebastopol, CA, 2008.
 - [28] GemTalk Systems, GemStone/S 64 Bit Programmer's Guide, GemTalk Systems, 2024. URL: <https://downloads.gemtalksystems.com/docs/GemStone64/3.7.x/GS64-ProgGuide-3.7/1-IntroToGemStone.htm>, version 3.7.x.
 - [29] Grail contributors, Grail, <https://github.com/GemTalk/Grail>, 2026. Accessed: 2026-08-14.
 - [30] Python Software Foundation, Python Language Reference, version 3.x, Python Software Foundation, 2026. URL: <https://www.python.org/>.
 - [31] G. van Rossum, Python tutorial, Technical Report CS-R9526, Centrum voor Wiskunde en Informatica (CWI), Amsterdam, 1995.
 - [32] Tree-sitter authors, Tree-sitter, 2026. URL: <https://tree-sitter.github.io/>.
 - [33] C. Béra, M. Denker, Towards a flexible pharo compiler, in: International Workshop on Smalltalk Technologies (IWST 2013), Annecy, France, 2013. URL: <https://inria.hal.science/hal-00862411v1/document>.
 - [34] Pharo contributors, Pharo, <https://github.com/pharo-project/pharo>, 2026. Accessed: 2026-08-14.